

CMPE 220- System Software

Program Layout & Execution

Project Report

BY

Sanjushree Golla

Gandhi Soumya A

Vignesh Jetty Ravi

Under the guidance of Prof. Ishie Eswar

6 May 2026

Github Repository

<https://github.com/SANJUSHREE19/220-System-Software>

Function Calls, Recursion, and Memory Layout

1. C Reference Function and Mapping to the Software CPU

In this project:

- The factorial program is implemented in our custom assembly program which is `factorial.asm` and runs on the Software CPU from Part 1 of our project.
- The assembly program initializes the input value 5 in register R0, which represents the function argument. It also sets up the stack pointer and frame pointer before invoking the FACTORIAL function.
- Once the function execution completes, the result of 5! (120) is stored in R0, and this value is then copied into R1 for final use.

This process establishes a clear correspondence between the behavior of the C-language implementation and its execution on the custom Software CPU.

Memory Layout for Factorial Program

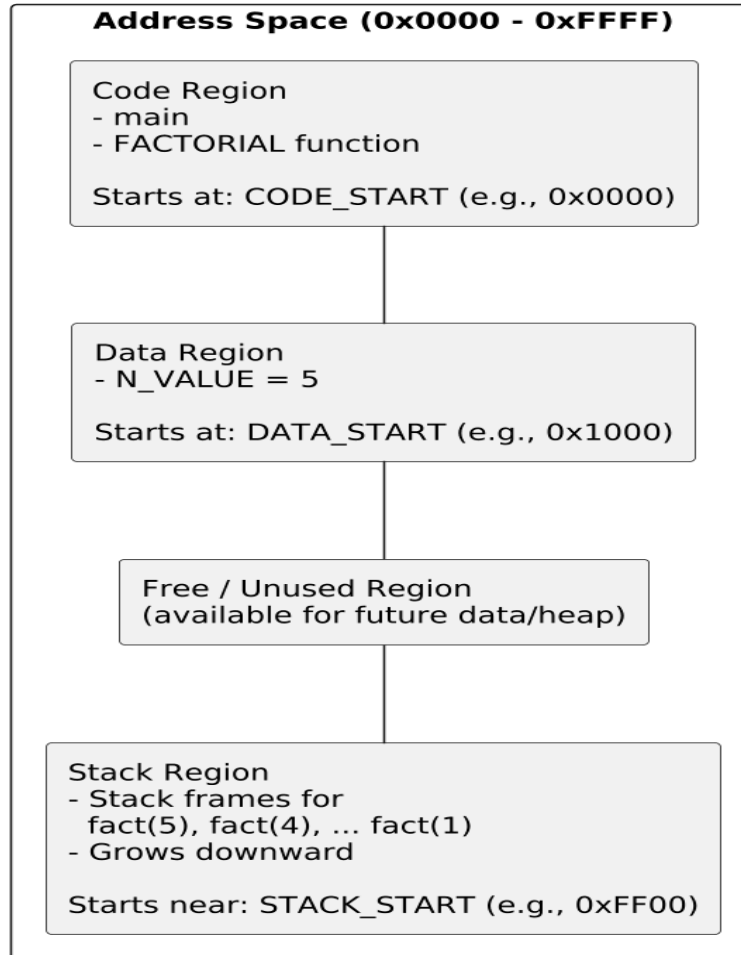


Figure 1 - Memory Layout for Factorial Program

Steps to download, compile and run the program

1. Download the project

Open the [GitHub repository link](#), in order to download the code you can do one of two things.

Option A - Download ZIP

- Click on the green **Code** button.
- Select **Download ZIP**.
- Extract or unzip the ZIP file to a folder on your computer (for example: CPU_Design).

Option B - Clone with Git

To clone from git, type the following code into terminal:

```
git clone https://github.com/SANJUSHREE19/220-System-Software
```

After this, you should see folders such as:

- In src/ - All C++ source code for the CPU, memory, assembler, and main driver.
- In asm/ - All example assembly programs (factorial.asm).
- README.md file with quick overview information and example commands.

2. Build (compile) the emulator

1. You need to have a C++ compiler installed (ex: g++) in order to be able to create the emulator.
2. From the root of the project folder (where src/ is located), run:

3. **Windows** - `g++ -std=c++11 -I./src src/*.cpp -o emulator`
macOS: `clang++ -std=c++11 -I./src src/*.cpp -o emulator` (or
g++ if installed)
4. Using the command line shown above in Windows, all files in `src/` are compiled into a single executable named `emulator.exe` in the same folder.

3. Run the programs and interpret the results

The emulator command-line interface is:

```
./emulator run <asm/factorial.asm>    or    .\emulator.exe  
run <factorial.asm>  
  
./emulator step <asm\factorial.asm>
```

Below we describe what is happening in detail with the result images for the factorial program.

3.1 Factorial program

Command

```
./emulator run asm/factorial.asm
```

Program behavior

The factorial program uses **recursion and the stack** to compute $N!$:

- The main code loads $N = 5$ into a register and sets up the initial frame pointer.
- It then executes `CALL FACTORIAL`.

- The FACTORIAL routine:
 - Starts with creating a stack frame using SP and FP,
 - Then checks the base case ($N \leq 1$) and returns 1 if it is true,
 - If not then it calls itself with $N-1$, and after the recursive call returns, multiplies the result by N .
- After all recursive calls are done the final result is left in R0 and copied into R1.

```

PS C:\SJSU\220\Prog-Layout-Exec-Team16> ./emulator run asm/factorial.asm
Assembly successful: 24 instructions. Data loaded starting at 0x1000
--- CPU Running ---
[PC=0x0000] LOADI | R0=0x0000 R1=0x0000 SP=0xFF00 FP=0xFF00 ZN=00
[PC=0x0002] LOADI | R0=0x0005 R1=0x0000 SP=0xFF00 FP=0xFF00 ZN=00
[PC=0x0004] ADD | R0=0x0005 R1=0x0000 SP=0xFF00 FP=0xFF00 ZN=00
[PC=0x0006] CALL | R0=0x0005 R1=0x0000 SP=0xFF00 FP=0xFF00 ZN=00
[PC=0x000C] PUSH | R0=0x0005 R1=0x0000 SP=0xFEFE FP=0xFF00 ZN=00
[PC=0x000E] ADD | R0=0x0005 R1=0x0000 SP=0xFEFC FP=0xFF00 ZN=00
[PC=0x0010] LOADI | R0=0x0005 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x0012] LOADI | R0=0x0005 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x0014] CMP | R0=0x0005 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x0016] JUMPEQ | R0=0x0005 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x0018] CMP | R0=0x0005 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x001A] JUMPEQ | R0=0x0005 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x001C] PUSH | R0=0x0005 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x001E] ADDI | R0=0x0005 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00
[PC=0x0020] CALL | R0=0x0004 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00
[PC=0x000C] PUSH | R0=0x0004 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00
[PC=0x000E] ADD | R0=0x0004 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00
[PC=0x0010] LOADI | R0=0x0004 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00
[PC=0x0012] LOADI | R0=0x0004 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00
[PC=0x0014] CMP | R0=0x0004 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00
[PC=0x0016] JUMPEQ | R0=0x0004 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00
[PC=0x0018] CMP | R0=0x0004 R1=0x0000 SP=0xFEFA FP=0xFEFC ZN=00

```

Figure 3.1.1 - Factorial program result

```
PS C:\SJSU\220\Prog-Layout-Exec-Team16> ./emulator run asm/factorial.asm
[PC=0x0026] JUMP | R0=0x0078 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x002A] ADD | R0=0x0078 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x002C] POP | R0=0x0078 R1=0x0000 SP=0xFEFC FP=0xFEFC ZN=00
[PC=0x002E] RET | R0=0x0078 R1=0x0000 SP=0xFEFE FP=0xFF00 ZN=00
[PC=0x0008] ADD | R0=0x0078 R1=0x0000 SP=0xFF00 FP=0xFF00 ZN=00
[PC=0x000A] HALT | R0=0x0078 R1=0x0078 SP=0xFF00 FP=0xFF00 ZN=00

--- CPU Halted ---

--- Registers ---
PC: 0x000c | IR: 0xf000 | Flags: 0x00
R0: 0x0078 | R1: 0x0078 | R2: 0x0000 | R3: 0x0005
R4: 0x0000 | R5: 0x0000 | FP: 0xff00 | SP: 0xff00

--- Memory Dump (1000 - 1013 [Hex]) ---
Addr | 00 01 02 03 04 05 06 07 | ASCII
-----
1000 | 05 00 00 00 00 00 00 00 | .....
1008 | 00 00 00 00 00 00 00 00 | .....
1010 | 00 00 00 00 00 00 00 00 | .....
-----
PS C:\SJSU\220\Prog-Layout-Exec-Team16> 
```

Figure 3.1.2 – Factorial program result

Team Contribution

Team Member	Contribution Area	Detailed Responsibilities
Sanjushree	Function Call Mechanism and Stack Frame Design	-Designed function calling mechanism for the custom CPU, including argument passing and return handling. -Implemented stack frame structure to support recursive calls. -Defined CALL and RET instruction behavior (saving/restoring return address using stack). -Verified correct operation of stack pointer (SP) and frame pointer (FP) through testing.
Gandhi Soumya	Recursive Function Implementation and Program Flow	-Developed recursive factorial program using the defined calling convention. -Created main driver program to initialize inputs and invoke functions. -Ensured correct control flow and recursion behavior in assembly programs.

Vignesh	Testing and Validation	- Tested recursive programs and verified correctness of outputs. - Debugged execution flow and stack behavior during function calls. - Validated integration between CPU, assembler, and program execution.
----------------	------------------------	---